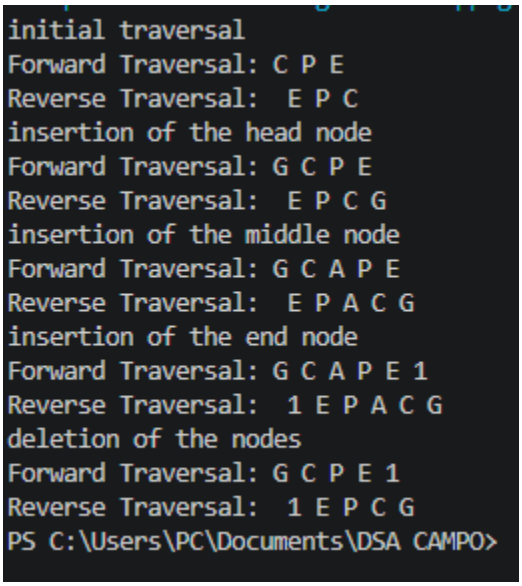
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	23/07/2026	**DATE SUBMITTED:**	23/07/2026
NAME:	Ramon Brendt D. Campo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

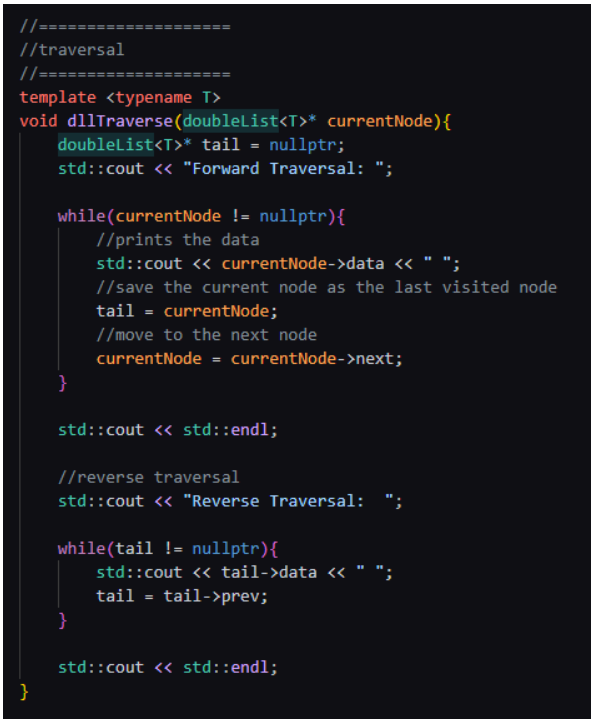
ACTIVITY NO. 3.1 **[Hands-on Activity 3.1 Linked Lists]**

1. PROCEDURE OUTPUT

Screenshot	
Discussion	This output right here is from a program made and structured using double linked list, pointers and heap memory. There are 5 operations showed in this program there is **traversal, insertion of the head node(starting node), general insertion, insertion of the end node (tail)** and the **deletion of the node.** The initial traversal has been activated in the program but the output itself is not showed thats because the traversal is the alignment and and connecting of the nodes, which for double linked list has 2 ways of accessing data which is straight from the head or the tail. Since its a double linked list it can go back and forth with nodes wherein a normal single linked list only has one way it can access data to The program first added the letters “C” “P” “E” into the

	nodes, showing how **forward traversal** works. The next output shows the opposite which is the **reverse traversal** where it reverses the order of the letters it also shows how the **insertion of the head node** which as the program shows stores the letter “G” in the head node, it also shows the forward and reverse traversal with the letter G included in the nodes. The next operation it outputed is the **general insertion** which in the program added the letter “A” in the middle. The next operation it showed is **insertion of the end node** which added the number “1” in the tail(end) of the node. Finally it showed the last operation which is the **Deletion of the node** which deletes all of the nodes once the program finishes so that it doesnt leave any “lost” data

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

```
//====================
//traversal
//====================
template <typename T>
void dllTraverse(doubleList<T>* currentNode){
    doubleList<T>* tail = nullptr;
    std::cout << "Forward Traversal: ";

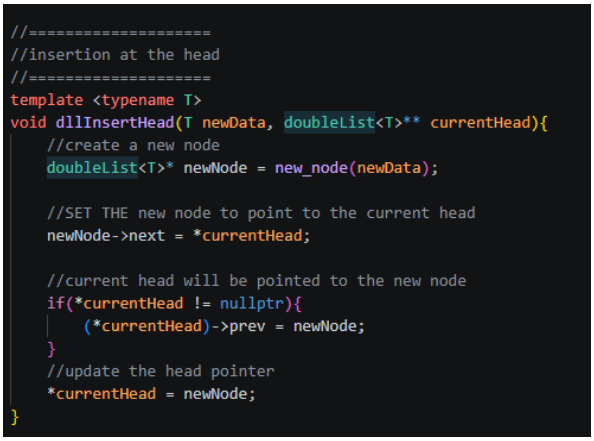
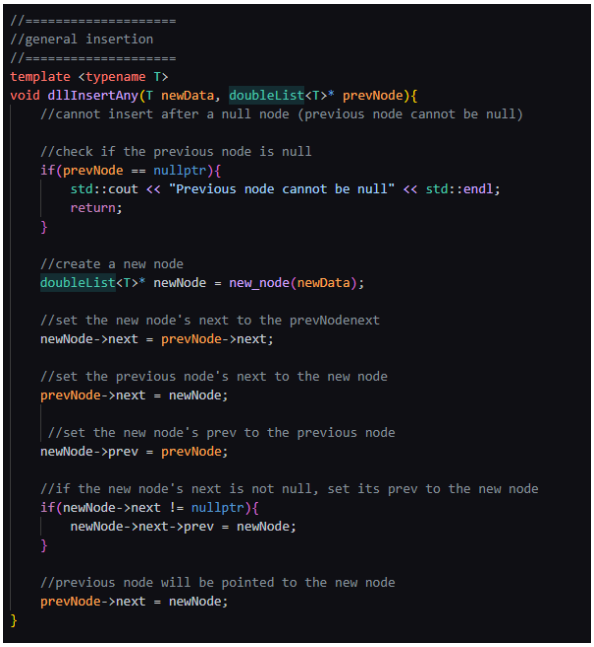
    while(currentNode != nullptr){
        //prints the data
        std::cout << currentNode->data << " ";
        //save the current node as the last visited node
        tail = currentNode;
        //move to the next node
        currentNode = currentNode->next;
    }

    std::cout << std::endl;

    //reverse traversal
    std::cout << "Reverse Traversal:  ";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl;
}
```

Insertion at the head	
Insertion at any part of the list	

```cpp
//====================
//insertion at the head
//====================
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);

    //SET THE new node to point to the current head
    newNode->next = *currentHead;

    //current head will be pointed to the new node
    if(*currentHead != nullptr){
        (*currentHead)->prev = newNode;
    }
    //update the head pointer
    *currentHead = newNode;
}
```

```cpp
//====================
//general insertion
//====================
template <typename T>
void dllInsertAny(T newData, doubleList<T>* prevNode){
    //cannot insert after a null node (previous node cannot be null)

    //check if the previous node is null
    if(prevNode == nullptr){
        std::cout << "Previous node cannot be null" << std::endl;
        return;
    }

    //create a new node
    doubleList<T>* newNode = new_node(newData);

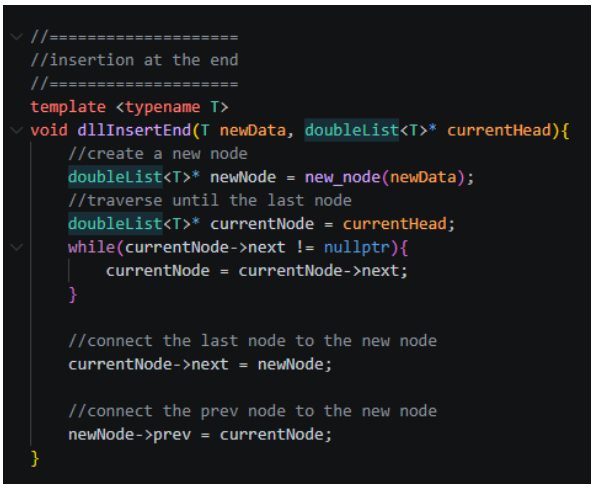
    //set the new node's next to the prevNodenext
    newNode->next = prevNode->next;

    //set the previous node's next to the new node
    prevNode->next = newNode;

     //set the new node's prev to the previous node
    newNode->prev = prevNode;

    //if the new node's next is not null, set its prev to the new node
    if(newNode->next != nullptr){
        newNode->next->prev = newNode;
    }

    //previous node will be pointed to the new node
    prevNode->next = newNode;
}
```

Insertion at the end	
Deletion of a node	

```
//====================
//insertion at the end
//====================
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);
    //traverse until the last node
    doubleList<T>* currentNode = currentHead;
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //connect the last node to the new node
    currentNode->next = newNode;

    //connect the prev node to the new node
    newNode->prev = currentNode;
}
```

```
//==================
//deletion
//==================

template <typename T>
void dllDelete(T findData, doubleList<T>** head){

    // Checks the list if its is empty, if it it ends the program-
    //-if its not it continues it
    if(*head == nullptr) return;

    //sets the current node to the head of the list
    doubleList<T>* currentNode = *head;


    //searches for the node and checks if it hasn't reach the end of the node
    //and if the data doenst match the data of the current node
    while(currentNode != nullptr && currentNode->data != findData){
        currentNode = currentNode->next;
    }

    // If data is not found
    if(currentNode == nullptr) return;

    // Delete the head node
    if(currentNode->prev == nullptr){
        *head = currentNode->next;

        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    }
    else{
        currentNode->prev->next = currentNode->next;

        if(currentNode->next != nullptr){
            currentNode->next->prev = currentNode->prev;
        }
    }

    delete currentNode;
}

#endif
```

Table 3-2. Code for the List Operations

A.	

	Source code	`//====================`
		`//creating a node`
		`//====================`
		`template <typename T>`
		`doubleList<T>* new_node(T newData){`
		`  //dynamically allocate memory for a new node`
		`  doubleList<T>* newNode = new doubleList<T>;`
		`  //store data in the node:`
		`  newNode->data = newData;`
		`  //since its a new node, it will point to null`
		`  newNode->next = nullptr;`
		`  newNode->prev = nullptr;`
		
		`  return newNode;`
		`}`

	Console	`initial traversal`
		`Forward Traversal: C P E`
		`Reverse Traversal:  E P C`

B.		

	Source code	`//====================`
		`//traversal`
		`//====================`
		`template <typename T>`
		`void dllTraverse(doubleList<T>* currentNode){`
		`  doubleList<T>* tail = nullptr;`
		`  std::cout << "Forward Traversal: ";`
		
		`  while(currentNode != nullptr){`
		`  //prints the data`
		`  std::cout << currentNode->data << " ";`
		`  //save the current node as the last visited node`
		`  tail = currentNode;`
		`  //move to the next node`
		`  currentNode = currentNode->next;`
		`  }`
		
		`  std::cout << std::endl;`

	Console	`Forward Traversal: C P E` `Reverse Traversal:  E P C`
C.		

Source code

```
//====================
//insertion at the head
//====================
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);

    //SET THE new node to point to the current head
    newNode->next = *currentHead;

    //current head will be pointed to the new node
    if(*currentHead != nullptr){
        (*currentHead)->prev = newNode;
    }
    //update the head pointer
    *currentHead = newNode;
}
```

Console

```
insertion of the head node
Forward Traversal: G C P E
Reverse Traversal:  E P C G
```

D.

Source code

```
//====================
//general insertion
//====================
template <typename T>
void dllInsertAny(T newData, doubleList<T>* prevNode){
    //cannot insert after a null node (previous node cannot be null)

    //check if the previous node is null
    if(prevNode == nullptr){
        std::cout << "Previous node cannot be null" << std::endl;
        return;
    }

    //create a new node
    doubleList<T>* newNode = new_node(newData);

    //set the new node's next to the prevNodenext
    newNode->next = prevNode->next;

    //set the previous node's next to the new node
    prevNode->next = newNode;

    //set the new node's prev to the previous node
    newNode->prev = prevNode;

    //if the new node's next is not null, set its prev to the new node
    if(newNode->next != nullptr){
        newNode->next->prev = newNode;
    }

    //previous node will be pointed to the new node
    prevNode->next = newNode;
}
```

Console

```
insertion of the middle node
Forward Traversal: G C A P E
Reverse Traversal:  E P A C G
```

E.

Source code

```
//====================
//insertion at the end
//====================
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);
    //traverse until the last node
    doubleList<T>* currentNode = currentHead;
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //connect the last node to the new node
    currentNode->next = newNode;

    //connect the prev node to the new node
    newNode->prev = currentNode;
}
```

Console

```
insertion of the end node
Forward Traversal: G C A P E 1
Reverse Traversal:  1 E P A C G
```

F.

Source code

```
//====================
//deletion
//====================

template <typename T>
void dllDelete(T findData, doubleList<T>** head){

    // Checks the list if its is empty, if it it ends the program-
    //-if its not it continues it
    if(*head == nullptr) return;

    //sets the current node to the head of the list
    doubleList<T>* currentNode = *head;


    //searches for the node and checks if it hasn't reach the end of the node
    //and if the data doenst match the data of the current node
    while(currentNode != nullptr && currentNode->data != findData){
        currentNode = currentNode->next;
    }

    // If data is not found
    if(currentNode == nullptr) return;

    // Delete the head node
    if(currentNode->prev == nullptr){
        *head = currentNode->next;

        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    }
    else{
        currentNode->prev->next = currentNode->next;

        if(currentNode->next != nullptr){
            currentNode->next->prev = currentNode->prev;
        }
    }

    delete currentNode;
}
#endif
```

Console	deletion of the nodes Forward Traversal: G C P E 1 Reverse Traversal: 1 E P C G

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot	Analysis
`void dllTraverse(doubleList<T>* currentNode){`	This is the traversal function where it combines the path of the nodes and makes sure it accesses it from both end and beginning
`void dllInsertHead(T newData, doubleList<T>** currentHead){`	This function inserts a value in the first/head node and stores it there
`void dllInsertAny(T newData, doubleList<T>* prevNode){`	This function stores a value anywhere in the nodes
`void dllInsertEnd(T newData, doubleList<T>* currentHead){`	This function stores a value in the end of the node
`void dllDelete(T findData, doubleList<T>** head){`	This function deletes the nodes once the program finishes

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SCREENSHOTS

MAIN.cpp

```cpp
#include <iostream>
#include <string>

#include "playlist.h"

int main() {

    //assigns the data type in the value t
    //creates the point variable head
    //head points to each song as an indicator, right now its set to nullptr

    playlist<std::string>* head = nullptr;
    int choice;

    //outputs the menu and choices
    do {
        std::cout << "1. Add Song\n2. Play All Songs\n3. Next Song\n4. Previous Song\n5. Remove Song\n6. Exit\nEnter choice: ";

        if (!(std::cin >> choice)) {
            std::cin.clear();
            std::cin.ignore(1000, '\n');
            std::cout << "invalid choice \n";
            continue;
        }

        //creates a menu for the user to choose and access the songs
        switch (choice) {
```

```
//====================
//add music
//====================
case 1:
{
    std::string song;

    std::cout << "Enter music name: ";
    std::cin.ignore(1000, '\n');
    std::getline(std::cin, song);

    //add the songname the head pointer to the function for it to execute
    addsong(song, &head);

    break;
}

//====================
//plays all song
//====================

case 2:
{
    std::cout << "=====playlist==== \n";

    playAll(head);

    break;
}

//====================
//next song
//====================
case 3:
{
    next_song(&head);
    break;
}
```

```
//====================
//previous song
//====================
case 4:
{
    prev_song(&head);
    break;
}

//====================
//removes current song
//====================
case 5:
{
    int numb;

    std::cout << "enter the song placement you want to delete: ";
    if (std::cin >> numb) {
        remove_song(numb, &head);
    } else {
        std::cin.clear();
        std::cin.ignore(1000, '\n');
        std::cout << "invalid choice \n";
    }

    break;
}

case 6:
{
    std::cout << "exiting playlist\n";
    break;
}

default:
{
    std::cout << "invalid choice \n";
    break;
}
```

```
            }
        }

    } while (choice != 6);

    // Free remaining nodes on exit
    while (head != nullptr) {
        remove_song(1, &head);
    }

    return 0;
}
```

PLAYLIST.h

```cpp
#ifndef playlist_H
#define playlist_H

#include <iostream>

template <typename T>
class playlist {
    public:
        T data;
        playlist<T>* next = nullptr;
        playlist<T>* prev = nullptr;
};

//====================
//newnode
//====================

template <typename T>
//This function creates a new node in the playlist
//this function and the variables in it stays throughout the entire program
playlist<T>* new_node(T newData) {

    playlist<T>* newNode = new playlist<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}

```

```cpp
//====================
//add music
//====================

template <typename T>
void addsong(T newData, playlist<T>** currenthead) {

    //holds the data of the new node

    //!note: when it calls the function new_node it can access the variables in the function
    //ex. data, next, prev because it is a pointer to the new node created in the function new_node

    //so everytime we create a new node we must call the function new_node to create a new node and store the data in it
    playlist<T>* newNode = new_node(newData);

    if (*currenthead == nullptr) {

        //it assigns next and prev in the new_node function to point to itself because it is the first node in the playlist
        //because of the if stat
        newNode->next = newNode;
        newNode->prev = newNode;

        *currenthead = newNode;
        return;
    }

    playlist<T>* tail = (*currenthead)->prev;

        playlist<T>* tail = (*currenthead)->prev;

        //connects to a new node which is the current head and the tail which is the last node in the playlist
        newNode->next = *currenthead;
        newNode->prev = tail;

        //connects the previous node which is tail to the new node which is the current head
        tail->next = newNode;
        (*currenthead)->prev = newNode;

        //holds the address of the new node which is the current head and assigns it to the current head pointer
        *currenthead = newNode;
    };

    //====================
    //next song
    //====================

    template <typename T>
    void next_song(playlist<T>** currenthead) {

        //to check if there are no songs in the playlist
        if (*currenthead == nullptr) {
            std::cout << "Playlist is empty. \n";
            return;
        }

        //points to the next song in the playlist
        *currenthead = (*currenthead)->next;

        std::cout << "Now playing: " << (*currenthead)->data << "\n";
    };

    //====================
    //previous song
    //====================
```

```cpp
template <typename T>
void prev_song(playlist<T>** currenthead) {

    if (*currenthead == nullptr) {
        std::cout << "Playlist is empty. \n";
        return;
    }

    //points to the previous song in the playlist
    *currenthead = (*currenthead)->prev;

    std::cout << "Now playing: " << (*currenthead)->data << "\n";
};

//==================
//plays all songs
//==================
template <typename T>
void playAll(playlist<T>* currentNode) {

    if (currentNode == nullptr) {
        std::cout << "Playlist is empty. \n";
        return;
    }

    //places the currentnode at the start by default
    playlist<T>* start = currentNode;
    int num = 1;

    //outputs the song and traverses to the songs
    do {
        std::cout << num << ". " << currentNode->data << std::endl;

        num++;
        currentNode = currentNode->next;
    } while (currentNode != start); //prints the data without going back to the start
                                    //(because currentnode is assigned to the start by default)
};
```

```cpp
//====================
// remove current song
//====================

template <typename T>
void remove_song(int pos, playlist<T>** currenthead) {

    if (*currenthead == nullptr) {
        std::cout << "Playlist is empty. Nothing to remove.\n";
        return;
    }

    if (pos < 1) {
        std::cout << "Invalid position.\n";
        return;
    }

    //the songToRemove is the pointer accesing the pointer of the currenthead
    playlist<T>* songToRemove = *currenthead;

    // if there is one song in the playlist it removes it
    //meaning if the next song/node is itself(meaning theres only one)it will delete the current node/song-
    //-and sets the currenthead pointer to null ptr
    if (songToRemove->next == songToRemove) {
        if (pos != 1) {
            std::cout << "Position out of bounds.\n";
            return;
        }
        std::cout << "Removed: " << songToRemove->data << "\n";
        delete songToRemove;
        *currenthead = nullptr;
        return;
    }
```

```
    //accesses the position given by the user in the main file
    for (int i = 1; i < pos; i++) {
        songToRemove = songToRemove->next;
        if (songToRemove == *currenthead) {
            std::cout << "Position out of bounds.\n";
            return;
        }
    }

    //creates a pointer called prevNode and nextNode and assigns it to the songToRemo
    //- pointer which indicates the next and previous songs will be removed
    playlist<T>* prevNode = songToRemove->prev;
    playlist<T>* nextNode = songToRemove->next;

    //assigns the next and previous song
    prevNode->next = nextNode;
    nextNode->prev = prevNode;

    // Move current head to the next song if we removed the original head node
    if (songToRemove == *currenthead) {
        *currenthead = nextNode;
    }

    //removes the chosen song to remove
    std::cout << "Removed: " << songToRemove->data << "\n";
    std::cout << "Now playing: " << (*currenthead)->data << "\n";

    // Free allocated memory
    delete songToRemove;
};

#endif
```

OUTPUT

```
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 1
Enter music name: man in the box
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 1
Enter music name: my own summer
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 1
Enter music name: byob
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 2
=====playlist====
1. byob
2. my own summer
3. man in the box
```

```
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 3
Now playing: my own summer
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 3
Now playing: man in the box
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 3
Now playing: byob
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 4
Now playing: man in the box
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
```

```
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 4
Now playing: my own summer
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 4
Now playing: byob
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 5
enter the song placement you want to delete: 3
Removed: man in the box
Now playing: byob
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 2
=====playlist====
1. byob
2. my own summer
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 7
invalid choice
```

```
1. Add Song
2. Play All Songs
3. Next Song
4. Previous Song
5. Remove Song
6. Exit
Enter choice: 6
exiting playlist
Removed: byob
Now playing: my own summer
Removed: my own summer
```

Conclusion

1. Summary of lessons learned?

-first of all I learned how looped linked list are created. Its created by making the currentHead (the pointer to indicate what part we are in the list) points to itself both the next and previous sides once there is a node, second I learned how to implement the linked list into an actual program and looking at it, it does make sense to have linked list or an area to store data in, its like similar to RAM sticks it needs to store a memory in it which im assuming it uses a similar type to what we just did here, which I think is pretty impactful when coding or saving important data. Lastly I know then functions of each line in the header file, because before this activity I really really struggled in learning how to structure the link list but right now I understand way more than I did last time

2. Analysis of the supplementary activity:

The program consists of 2 files the main.cpp file and the header file. The main.cpp file is where the functions in the header file is called, there are some variables that are in the main.cpp file for example head and the string data type but all of them are defined in the header file. Lets go first with the header file, the header file has all the functions that I previously learned like **insertions, deletions, adding of node, displaying all etc.** but theres a different approach to this. Since we are creating a looped double linked list we need to figure out how to change the pointer of the tail and head to instead point to each other respectfully, which is implemented on the addsong function as previously mentioned. Now for the main file I had to use a switch case statement for it to create a menu, to access the double linked list and its values, I also added a “invalid input” indication to both the header and main.cpp file b

3. Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

This activity is probably one of the stressful things I did and its gonna get harder from this. The mixture being inexperienced in coding, not knowing how to loop, bad at troubleshooting, not knowing how to implement the double linked list and overall just the lack of knowledge in linked list plus the struggle of time pressure, makes this activity feel like a final project. But I still manage to endure and finished this activity. I really thing my room for improvement is code implementation, I know how linked list works I just get confused on how to implement in code, I also need to improve in learning things fast and just overall practice programming and linked list in order to get a better grasp on programming in the future, but in the end I know how linked lists work way better now,

wouldn't say the best but its atleast an improvement

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **To explain variables, give examples and indicate errors**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used it to explain how each function works in the double linked list and ask it to give samples.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.